class13

Aiden Quisenberry(PID: A19083392)

Table of contents

Background 4
Data Import 4
Check on metadata counts correspondance 5
Analysis Plan… 6
Log2 units and fold change 8
Remove zero count genes 9
DESeq analysis 9
Volcano plot 11
Add some plot annotation 12
Add Annoation Data 12
Pathway analysis 15

```r
library(DESeq2)
```

```
Warning: package 'DESeq2' was built under R version 4.5.2

Loading required package: S4Vectors

Warning: package 'S4Vectors' was built under R version 4.5.2

Loading required package: stats4

Loading required package: BiocGenerics

Loading required package: generics
```

```
Attaching package: 'generics'

The following objects are masked from 'package:base':

    as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
    setequal, union


Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

    IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

    anyDuplicated, aperm, append, as.data.frame, basename, cbind,
    colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
    get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
    order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
    rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
    unsplit, which.max, which.min


Attaching package: 'S4Vectors'

The following object is masked from 'package:utils':

    findMatches

The following objects are masked from 'package:base':

    expand.grid, I, unname

Loading required package: IRanges

Warning: package 'IRanges' was built under R version 4.5.2

Loading required package: GenomicRanges
```

```
Warning: package 'GenomicRanges' was built under R version 4.5.2

Loading required package: Seqinfo

Loading required package: SummarizedExperiment

Loading required package: MatrixGenerics

Loading required package: matrixStats


Attaching package: 'MatrixGenerics'

The following objects are masked from 'package:matrixStats':

    colAlls, colAnyNAs, colAnys, colAvgsPerRowSet, colCollapse,
    colCounts, colCummaxs, colCummins, colCumprods, colCumsums,
    colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,
    colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,
    colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
    colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
    colWeightedMeans, colWeightedMedians, colWeightedSds,
    colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgsPerColSet,
    rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
    rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
    rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
    rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
    rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
    rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
    rowWeightedSds, rowWeightedVars

Loading required package: Biobase

Warning: package 'Biobase' was built under R version 4.5.2

Welcome to Bioconductor

    Vignettes contain introductory material; view with
    'browseVignettes()'. To cite Bioconductor, see
    'citation("Biobase")', and for packages 'citation("pkgname")'.
```

```
Attaching package: 'Biobase'

The following object is masked from 'package:MatrixGenerics':

    rowMedians

The following objects are masked from 'package:matrixStats':

    anyMissing, rowMedians
```

Background

Today we will perform an RNASeq analysis on the effects of dexamethasone (dex), a common steroid, on airway smooth muscle (ASM) cell lines.

Data Import

We need two things for this analysis:

- **countData**: a table with genes as rows and samples/experiments as columns,
- **colData**: metadata about the columns (i.e. samples) in the main countData object.

```r
counts <- read.csv("airway_scaledcounts.csv", row.names = 1)
metadata <- read.csv("airway_metadata.csv")
```

Let's have a wee peak at these two objects:

```r
metadata
```

```
          id     dex celltype     geo_id
1 SRR1039508 control   N61311 GSM1275862
2 SRR1039509 treated   N61311 GSM1275863
3 SRR1039512 control  N052611 GSM1275866
4 SRR1039513 treated  N052611 GSM1275867
5 SRR1039516 control  N080611 GSM1275870
6 SRR1039517 treated  N080611 GSM1275871
7 SRR1039520 control  N061011 GSM1275874
8 SRR1039521 treated  N061011 GSM1275875
```

```r
head(counts)
```

```
                SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR1039516
ENSG00000000003        723        486        904        445       1170
ENSG00000000005          0          0          0          0          0
ENSG00000000419        467        523        616        371        582
ENSG00000000457        347        258        364        237        318
ENSG00000000460         96         81         73         66        118
ENSG00000000938          0          0          1          0          2
                SRR1039517 SRR1039520 SRR1039521
ENSG00000000003       1097        806        604
ENSG00000000005          0          0          0
ENSG00000000419        781        417        509
ENSG00000000457        447        330        324
ENSG00000000460         94        102         74
ENSG00000000938          0          0          0
```

Check on metadata counts correspondance

We need to check that the metadata matches the samples in our count data.

```r
ncol(counts) == nrow(metadata)
```

```
[1] TRUE
```

can use `all()` Function to check that all data line up and match. True means they do, meaning they correspond to the same vector.

```r
all(colnames(counts) == metadata$id)
```

```
[1] TRUE
```

> Q1. How many genes are in this dataset?

```r
nrow(counts)
```

```
[1] 38694
```

38694

Q2. How many "control" samples are in this dataset?

```
sum(metadata$dex == "control")
```

```
[1] 4
```

4

Analysis Plan...

We have 4 replicates per condition ("control" and "treated"). We want to compare the control vs the treated to see which genes expression levels change when we have the drug present.

Step 1. Find which columns in `counts` correspond to "control" samples.

Step 2. Select these columns.

Step 3. Calculate an average value for each gene (i.e. each row).

```
# The indices (i.e positions) that are "control"
ctrl.inds <- metadata$dex == "control"
```

```
ctrl.counts <- counts[,ctrl.inds]
```

```
ctrl.means <- rowMeans(ctrl.counts)
head(ctrl.means )
```

```
ENSG00000000003 ENSG00000000005 ENSG00000000419 ENSG00000000457 ENSG00000000460
         900.75            0.00          520.50          339.75           97.25
ENSG00000000938
           0.75
```

Q. Do the same for "treated" samples - find the mean count value per gene.

```
trtd.inds <- metadata$dex == "treated"
trtd.counts <-  counts[,trtd.inds]
trtd.means <- rowMeans(trtd.counts)
#Can also do one line:rowMeans( counts[,metadata$dex == "treated"])
```

Let's put these two mean values into a new data.frame `meancounts` for easy book-keeping and plotting.

```r
meancounts <- data.frame(ctrl.means, trtd.means)
head(meancounts)
```

```
                ctrl.means trtd.means
ENSG00000000003     900.75     658.00
ENSG00000000005       0.00       0.00
ENSG00000000419     520.50     546.00
ENSG00000000457     339.75     316.50
ENSG00000000460      97.25      78.75
ENSG00000000938       0.75       0.00
```

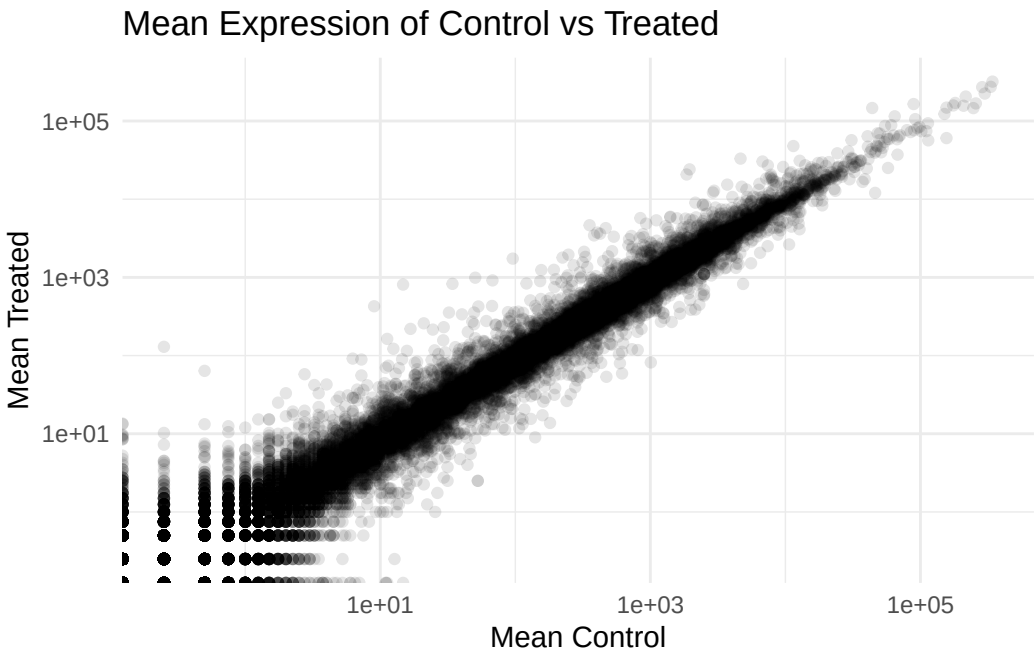
Q. make a ggplot of average counts of control vs treated.

```r
library(ggplot2)

ggplot(meancounts, aes(ctrl.means, trtd.means)) +
  geom_point(alpha = .1) +
  theme_minimal() +
  labs(title = "Mean Expression of Control vs Treated", x = "Mean Control", y = "Mean Treated
  scale_x_log10() +
  scale_y_log10()
```

```
Warning in scale_x_log10(): log-10 transformation introduced infinite values.

Warning in scale_y_log10(): log-10 transformation introduced infinite values.
```



Log2 units and fold change

If we consider "treated"/"control" counts we will get a number that tells us the change.

```
log2(20/20) # No change
```

```
[1] 0
```

```
log2(40/20) # A doubling in the treated vs control
```

```
[1] 1
```

```
log2(10/20) # A
```

```
[1] -1
```

> Q. Add a new column called `log2fc` for log2fold change of treated/control to our `meancounts` object.

```r
meancounts$log2fc <- log2(meancounts$trtd.means/meancounts$ctrl.means)
head(meancounts)
```

```
                ctrl.means trtd.means      log2fc
ENSG00000000003     900.75     658.00 -0.45303916
ENSG00000000005       0.00       0.00         NaN
ENSG00000000419     520.50     546.00  0.06900279
ENSG00000000457     339.75     316.50 -0.10226805
ENSG00000000460      97.25      78.75 -0.30441833
ENSG00000000938       0.75       0.00        -Inf
```

Remove zero count genes

Typically we would not consider zero count genes - as we have no data about them and they should be excluded from further consideration. These lead to "funky" log2 fold change values (e.g. division by zero errors etc…)

DESeq analysis

We are missing any measure of significiance from the work we have so far. Let's do this properly with the **DESeq2** package

```r
library(DESeq2)
```

The DESeq2 package, like many bioconductor packages, wants its input in a very specific way - a data structure setup with all the info it needs for the calculation.

```r
dds <- DESeqDataSetFromMatrix(countData = counts,
                        colData = metadata,
                        design = ~dex)
```

```
converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
design formula are characters, converting to factors
```

The main function in this package is called `DESeq()`, it will the full analysis for us on our `dds` input object:

```r
dds <-DESeq(dds)
```

```
estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing
```

Extract our results:

```r
res <- results(dds)
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 6 rows and 6 columns
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195     -0.3507030  0.168246 -2.084470 0.0371175
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160      0.2061078  0.101059  2.039475 0.0414026
ENSG00000000457 322.664844      0.0245269  0.145145  0.168982 0.8658106
ENSG00000000460  87.682625     -0.1471420  0.257007 -0.572521 0.5669691
ENSG00000000938   0.319167     -1.7322890  3.493601 -0.495846 0.6200029
                     padj
                <numeric>
ENSG00000000003  0.163035
ENSG00000000005        NA
ENSG00000000419  0.176032
ENSG00000000457  0.961694
ENSG00000000460  0.815849
ENSG00000000938        NA
```

Volcano plot

A useful and ubiquitous plot used to visualize the log2 fold change values vs adjusted p-values.

> Q. Use ggplot to make a first version "volcano plot" of Log2FoldChange vs padj

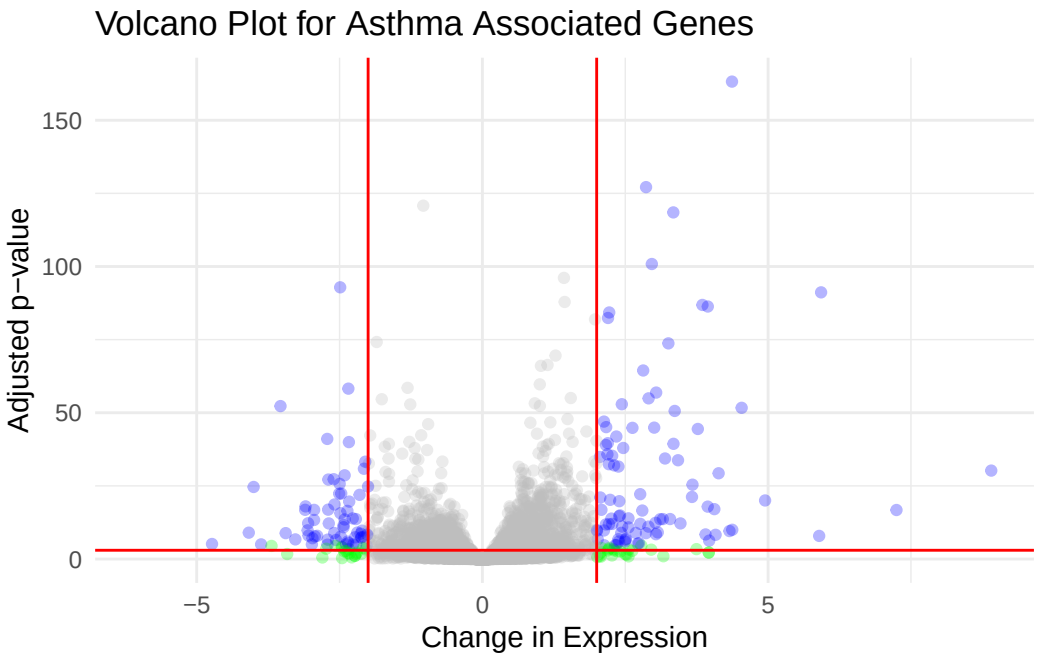
```
library(ggplot2)

mycols <- rep("gray", nrow(res))
mycols[abs(res$log2FoldChange) > 2 ]  <- "green"

inds <- (res$padj < 0.01) & (abs(res$log2FoldChange) > 2 )
mycols[ inds ] <- "blue"

ggplot(res) +
  aes(log2FoldChange,
       -log(padj)) +
        geom_point(col = mycols, alpha = .3) +
  geom_vline(xintercept = c(-2,+2), col = "red") +
  geom_hline(yintercept = -log(.05), col = "red") +
  labs(title="Volcano Plot for Asthma Associated Genes", x = "Change in Expression", y = "Adj
  theme_minimal()
```

```
Warning: Removed 23549 rows containing missing values or values outside the scale range
(`geom_point()`).
```



This is not very useful

Add some plot annotation

> Q. Add color to the points (genes) we care about, nice axis labels, a useful title and a nice theme.

#Save our results to a CSV file

```
write.csv(res, file="results.csv")
```

Add Annoation Data

To make sense of our results we need to know what the differentially expressed genes are what biological pathways and processes they are involved in.

```
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 6 rows and 6 columns
```

```
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195     -0.3507030  0.168246 -2.084470 0.0371175
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160      0.2061078  0.101059  2.039475 0.0414026
ENSG00000000457 322.664844      0.0245269  0.145145  0.168982 0.8658106
ENSG00000000460  87.682625     -0.1471420  0.257007 -0.572521 0.5669691
ENSG00000000938   0.319167     -1.7322890  3.493601 -0.495846 0.6200029
                     padj
                 <numeric>
ENSG00000000003  0.163035
ENSG00000000005        NA
ENSG00000000419  0.176032
ENSG00000000457  0.961694
ENSG00000000460  0.815849
ENSG00000000938        NA
```

Let's start by mapping our ENSEMBLE ids to the more conventional gene SYMBOL.

We will use two bioconductor packages for this "mapping" **AnnotationDbi** and **org.Hs.eg.db**

We will first need to install these from bioconductor with `BiocManager::install("package_name")`

```r
library(AnnotationDbi)
library(org.Hs.eg.db)
```

```r
columns(org.Hs.eg.db)
```

```
 [1] "ACCNUM"       "ALIAS"        "ENSEMBL"      "ENSEMBLPROT"  "ENSEMBLTRANS"
 [6] "ENTREZID"     "ENZYME"       "EVIDENCE"     "EVIDENCEALL"  "GENENAME"
[11] "GENETYPE"     "GO"           "GOALL"        "IPI"          "MAP"
[16] "OMIM"         "ONTOLOGY"     "ONTOLOGYALL"  "PATH"         "PFAM"
[21] "PMID"         "PROSITE"      "REFSEQ"       "SYMBOL"       "UCSCKG"
[26] "UNIPROT"
```

`mapIds()` will translate a dataset's gene id's to a different bioinformatics format (i.e ENSEMBL -> SYMBOL)

```r
res$symbol <- mapIds(org.Hs.eg.db,
       keys = rownames(res), #Our ids
       keytype = "ENSEMBL", # Their Format
       column = "SYMBOL") # What I want to translate to
```

```
'select()' returned 1:many mapping between keys and columns
```

> Q. Can you add "GENENAME" and "ENTREZID" as new columns to the `res` as "name" and "entrez"

```r
res$name <- mapIds(org.Hs.eg.db,
       keys = rownames(res), #Our ids
       keytype = "ENSEMBL", # Their Format
       column = "GENENAME") # What I want to translate to
```

```
'select()' returned 1:many mapping between keys and columns
```

```r
res$entrez <- mapIds(org.Hs.eg.db,
       keys = rownames(res), #Our ids
       keytype = "ENSEMBL", # Their Format
       column = "ENTREZID") # What I want to translate to
```

```
'select()' returned 1:many mapping between keys and columns
```

```r
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 6 rows and 9 columns
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195     -0.3507030  0.168246 -2.084470 0.0371175
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160      0.2061078  0.101059  2.039475 0.0414026
ENSG00000000457 322.664844      0.0245269  0.145145  0.168982 0.8658106
ENSG00000000460  87.682625     -0.1471420  0.257007 -0.572521 0.5669691
ENSG00000000938   0.319167     -1.7322890  3.493601 -0.495846 0.6200029
                     padj      symbol                   name      entrez
                 <numeric> <character>            <character> <character>
ENSG00000000003  0.163035      TSPAN6          tetraspanin 6        7105
```

```
ENSG00000000005        NA        TNMD            tenomodulin       64102
ENSG00000000419  0.176032        DPM1 dolichyl-phosphate m..        8813
ENSG00000000457  0.961694       SCYL3 SCY1 like pseudokina..       57147
ENSG00000000460  0.815849       FIRRM FIGNL1 interacting r..       55732
ENSG00000000938        NA         FGR FGR proto-oncogene, ..        2268
```

```
write.csv(res, #File Data
          file="results_annotated.csv") #File name (saved to current session directory)
```

Pathway analysis

Now we know the gen names (gene symbols) and their entrez IDs we can find out what [athways they are involved in. This is called "pathway analysis" or "gen set enrichment"

We will use the **gage** package

```
library(gage)
library(gageData)
library(pathview)
```

```
data(kegg.sets.hs)
head(kegg.sets.hs, 2)
```

```
$`hsa00232 Caffeine metabolism`
[1] "10"   "1544" "1548" "1549" "1553" "7498" "9"

$`hsa00983 Drug metabolism - other enzymes`
 [1] "10"     "1066"   "10720"  "10941"  "151531" "1548"   "1549"   "1551"
 [9] "1553"   "1576"   "1577"   "1806"   "1807"   "1890"   "221223" "2990"
[17] "3251"   "3614"   "3615"   "3704"   "51733"  "54490"  "54575"  "54576"
[25] "54577"  "54578"  "54579"  "54600"  "54657"  "54658"  "54659"  "54963"
[33] "574537" "64816"  "7083"   "7084"   "7172"   "7363"   "7364"   "7365"
[41] "7366"   "7367"   "7371"   "7372"   "7378"   "7498"   "79799"  "83549"
[49] "8824"   "8833"   "9"      "978"
```

To run our pathway analysis we will use the **gage()** function. It wants two main inputs: a vector of importance (in our case, the log2fold change values); and the gene sets to check overlap for.

```r
foldchanges <- res$log2FoldChange
names(foldchanges) <- res$symbol
head(foldchanges)
```

```
     TSPAN6        TNMD        DPM1       SCYL3       FIRRM         FGR 
-0.35070302          NA  0.20610777  0.02452695 -0.14714205 -1.73228897 
```

KEGG speaks entrez (i.e. uses ENTREZID format) not gene symbol format

```r
names(foldchanges) <-  res$entrez
```

```r
keggres = gage(foldchanges, gsets=kegg.sets.hs)
```

```r
head(keggres$less, 5)
```

```
                                                       p.geomean stat.mean
hsa05332 Graft-versus-host disease                    0.0004250461 -3.473346
hsa04940 Type I diabetes mellitus                     0.0017820293 -3.002352
hsa05310 Asthma                                       0.0020045888 -3.009050
hsa04672 Intestinal immune network for IgA production 0.0060434515 -2.560547
hsa05330 Allograft rejection                          0.0073678825 -2.501419
                                                             p.val      q.val
hsa05332 Graft-versus-host disease                    0.0004250461 0.09053483
hsa04940 Type I diabetes mellitus                     0.0017820293 0.14232581
hsa05310 Asthma                                       0.0020045888 0.14232581
hsa04672 Intestinal image network for IgA production 0.0060434515 0.31387180
hsa05330 Allograft rejection                          0.0073678825 0.31387180
                                                      set.size         exp1
hsa05332 Graft-versus-host disease                          40 0.0004250461
hsa04940 Type I diabetes mellitus                           42 0.0017820293
hsa05310 Asthma                                             29 0.0020045888
hsa04672 Intestinal immune network for IgA production       47 0.0060434515
hsa05330 Allograft rejection                                36 0.0073678825
```

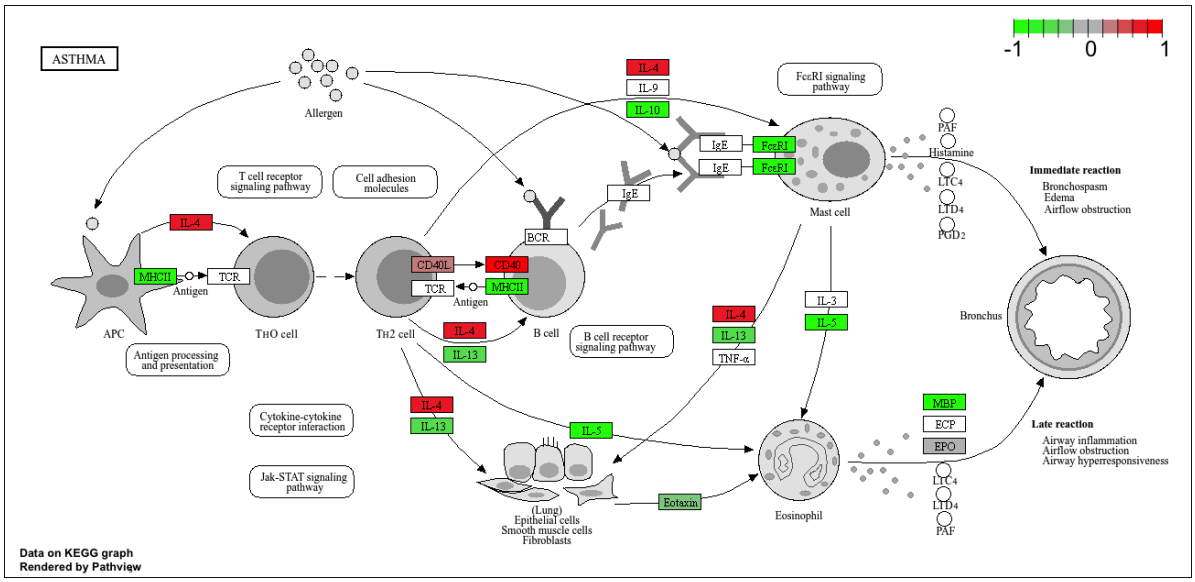
Let's make a figure of one of these pathways with our DEGs highlighted:

```r
pathview(foldchanges, pathway.id = "hsa05310")
```

```
'select()' returned 1:1 mapping between keys and columns
```

```
Info: Working in directory /Users/aidenquisenberry/Desktop/Biology Series/Bioinformatics/Rstu
```

```
Info: Writing image file hsa05310.pathview.png
```



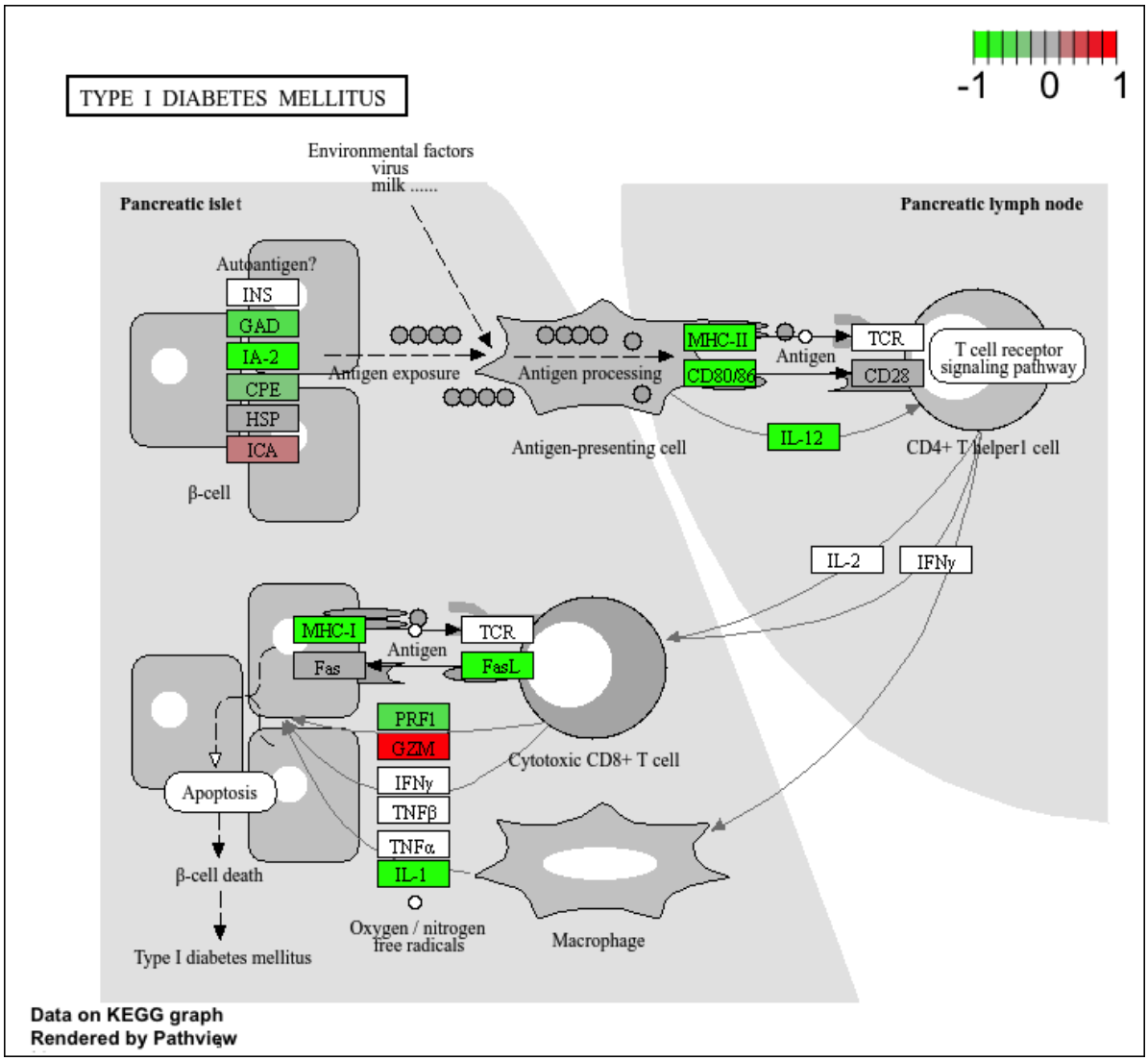
Q. Generate and insert a pathway figure for "Graft-versus-host disease" and "Type I diabetes"?

```
pathview(foldchanges, pathway.id = "hsa04940")
```

```
'select()' returned 1:1 mapping between keys and columns
```

```
Info: Working in directory /Users/aidenquisenberry/Desktop/Biology Series/Bioinformatics/Rstu
```

```
Info: Writing image file hsa04940.pathview.png
```


TYPE I DIABETES MELLITUS
-1
0
1
Environmental factors
virus
milk .....
Pancreatic islet
Pancreatic lymph node
Autoantigen?
INS
GAD
IA-2
CPE
HSP
ICA
β-cell
Antigen exposure
Antigen processing
Antigen-presenting cell
MHC-II
CD80/86
Antigen
TCR
CD28
T cell receptor signaling pathway
IL-12
CD4+ T helper1 cell
IL-2
IFNγ
MHC-I
Fas
Antigen
TCR
FasL
PRF1
GZM
IFNγ
TNFβ
TNFα
IL-1
Cytotoxic CD8+ T cell
Apoptosis
β-cell death
Type I diabetes mellitus
Oxygen / nitrogen
free radicals
Macrophage
Data on KEGG graph
Rendered by Pathview